

03. Semi-Supervised Learning: Self-Training

Algorithm implemented: Self-Training Classifier (iterative pseudo-labeling) with an RBF-kernel SVC base estimator.

Dataset: Breast Cancer Wisconsin - only 15% of the training labels are kept; the rest are treated as unlabeled.

```
# ---- Suppress a known sklearn deprecation notice for SVC(probability=True) ----
import warnings
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Dataset + train/test utilities ----
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# ---- Semi-supervised self-training wrapper ----
from sklearn.semi_supervised import SelfTrainingClassifier
# ---- Base estimators compared inside self-training ----
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
# ---- Evaluation metrics ----
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
confusion_matrix, classification_report

# Aesthetics setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# The installed scikit-Learn still supports SVC(probability=True) but warns; silence that specific notice
warnings.filterwarnings("ignore", message="The `probability` parameter was deprecated", category=FutureWarning)
np.random.seed(42)
print("Semi-supervised libraries imported successfully!")

# ---- Load the Breast Cancer Wisconsin dataset ----
data = load_breast_cancer()
X_full = data.data          # 30 continuous diagnostic features
y_full = data.target        # 0 = malignant, 1 = benign
feature_names = data.feature_names
target_names = data.target_names

# ---- Train / Test split (stratified to preserve class balance) ----
X_train_full, X_test, y_train_full, y_test = train_test_split(
    X_full, y_full, test_size=0.30, random_state=42, stratify=y_full
)

# ---- Standardize features (fit on train only to avoid test Leakage) ----
scaler = StandardScaler()
X_train_full_scaled = scaler.fit_transform(X_train_full)
X_test_scaled = scaler.transform(X_test)

# ---- Simulate the semi-supervised setting: hide 85% of the training labels ----
unlabeled_ratio = 0.85
rng = np.random.RandomState(42)
random_unlabeled_points = rng.rand(len(y_train_full)) < unlabeled_ratio # True -> mask

y_train_semi = np.copy(y_train_full)
y_train_semi[random_unlabeled_points] = -1 # -1 is the scikit-Learn convention for "unlabeled"

n_labeled = np.sum(y_train_semi != -1)
n_unlabeled = np.sum(y_train_semi == -1)

print(f"Total Training Samples: {len(y_train_full)}")
print(f" - Labeled Samples (15%): {n_labeled}")
print(f" - Unlabeled Samples (85%): {n_unlabeled}")
print(f"Holdout Test Samples: {len(y_test)}")
```

```
Semi-supervised libraries imported successfully!
Total Training Samples: 398
 - Labeled Samples (15%): 61
 - Unlabeled Samples (85%): 337
Holdout Test Samples: 171
```

Step-by-Step Algorithm (Self-Training)

1. **Input:** labeled set L , unlabeled pool U , base classifier f , confidence threshold τ .
2. Train f on the current labeled set L .
3. Predict class probabilities for every sample in U .
4. Add predictions with max probability $\geq \tau$ to L as pseudo-labels; remove them from U .
5. Repeat 2-4 until no new pseudo-labels are added or the iteration limit is reached.
6. Return the final classifier; evaluate once on the untouched test set.

Pseudocode (Self-Training Classifier)

```
L = labeled set          # 15% of training data
U = unlabeled pool       # 85% of training data (labels hidden)
repeat until no new confident labels or max_iter reached:
    f = train(base_estimator, L)          # e.g. SVC or Random Forest
    P = f.predict_proba(U)                # class probabilities for each unlabeled
point
    S = { (x, argmax P(x)) : max P(x) >= threshold }  # confident pseudo-labels
    L = L union S                                  # augment labeled set
    U = U minus S                                  # remove newly labeled points
return f
```

Key hyperparameter: the confidence threshold τ controls the purity/quantity trade-off of pseudo-labels.

```
# ---- Experiment 1: Self-Training with an RBF-kernel SVC ----

# 1. Supervised baseline trained ONLY on the ~60 Labeled samples
mask_labeled = (y_train_semi != -1)          # boolean mask of the labeled subset
X_labeled = X_train_full_scaled[mask_labeled]
y_labeled = y_train_semi[mask_labeled]

base_svc = SVC(kernel='rbf', probability=True, C=1.0, random_state=42)
base_svc.fit(X_labeled, y_labeled)
y_pred_baseline_svc = base_svc.predict(X_test_scaled)
acc_baseline_svc = accuracy_score(y_test, y_pred_baseline_svc)
f1_baseline_svc = f1_score(y_test, y_pred_baseline_svc)

# 2. Semi-supervised self-training: base SVC + iterative pseudo-labeling above threshold 0.80
self_training_svc = SelfTrainingClassifier(
    estimator=SVC(kernel='rbf', probability=True, C=1.0, random_state=42),
    threshold=0.80,          # only accept pseudo-labels with confidence >= 0.80
    criterion='threshold',   # use the confidence-threshold selection rule
    max_iter=15,
    verbose=True             # print how many labels are added per iteration
)
self_training_svc.fit(X_train_full_scaled, y_train_semi)
y_pred_semi_svc = self_training_svc.predict(X_test_scaled)
acc_semi_svc = accuracy_score(y_test, y_pred_semi_svc)
f1_semi_svc = f1_score(y_test, y_pred_semi_svc)

# 3. Fully supervised "ceiling": train on 100% of the ground-truth Labels
full_svc = SVC(kernel='rbf', probability=True, C=1.0, random_state=42)
full_svc.fit(X_train_full_scaled, y_train_full)
y_pred_full_svc = full_svc.predict(X_test_scaled)
acc_full_svc = accuracy_score(y_test, y_pred_full_svc)
f1_full_svc = f1_score(y_test, y_pred_full_svc)

# ---- Compare the three settings ----
print(f"\n--- SVC Evaluation Results ---")
print(f"Supervised Baseline (15% labeled): Accuracy = {acc_baseline_svc:.4f}, F1 = {f1_baseline_svc:.4f}")
print(f"Self-Training (Semi-supervised): Accuracy = {acc_semi_svc:.4f}, F1 = {f1_semi_svc:.4f}")
print(f"Fully Supervised Ceiling (100%): Accuracy = {acc_full_svc:.4f}, F1 = {f1_full_svc:.4f}")
```

End of iteration 1, added 292 new labels.
End of iteration 2, added 22 new labels.
End of iteration 3, added 5 new labels.
End of iteration 4, added 2 new labels.

```
--- SVC Evaluation Results ---
Supervised Baseline (15% labeled): Accuracy = 0.9298, F1 = 0.9469
Self-Training (Semi-supervised): Accuracy = 0.9415, F1 = 0.9554
Fully Supervised Ceiling (100%): Accuracy = 0.9766, F1 = 0.9813
```

```
# ---- Inspect how self-training converged ----
# n_iter_ : number of self-training iterations performed
# termination_condition_ : why training stopped (e.g. 'no_change' = no new confident labels)
# transduction_ : final labels for every training sample (-1 = still unlabeled)
n_iter = self_training_svc.n_iter_
termination_condition = self_training_svc.termination_condition_
labeled_final_mask = (self_training_svc.transduction_ != -1)

print(f"Self-Training Convergence Summary:")
print(f" - Iterations completed: {n_iter}")
print(f" - Termination condition: {termination_condition}")
print(f" - Total samples labeled after self-training: {np.sum(labeled_final_mask)} / {len(y_train_full)}")
print(f" - Pseudo-labels newly added: {np.sum(labeled_final_mask) - n_labeled}")
```

```
Self-Training Convergence Summary:
- Iterations completed: 5
- Termination condition: no_change
- Total samples labeled after self-training: 382 / 398
- Pseudo-labels newly added: 321
```

Results

Compare the 15%-labeled baseline, self-training, and the fully supervised ceiling.

```
# ---- Compact comparison table (SVC) ----
summary_data = [
    {"Model": "SVC (Baseline 15% Labeled)", "Accuracy": acc_baseline_svc, "F1-Score": f1_baseline_svc,
     "Training Samples": n_labeled},
    {"Model": "SVC (Self-Training Semi-Supervised)", "Accuracy": acc_semi_svc, "F1-Score": f1_semi_svc,
     "Training Samples": np.sum(self_training_svc.transduction_ != -1)},
    {"Model": "SVC (Fully Supervised Ceiling 100%)", "Accuracy": acc_full_svc, "F1-Score": f1_full_svc,
     "Training Samples": len(y_train_full)},
]
summary_df = pd.DataFrame(summary_data).set_index("Model")
display(summary_df.style.highlight_max(subset=['Accuracy', 'F1-Score'], color='lightgreen'))
```

	Accuracy	F1-Score	Training Samples
Model			
SVC (Baseline 15% Labeled)	0.929825	0.946903	61
SVC (Self-Training Semi-Supervised)	0.941520	0.955357	382
SVC (Fully Supervised Ceiling 100%)	0.976608	0.981308	398